# MCP Server Vetting Checklist ("Prüfraster")

Apply this to **every server you shortlist**. Most answers are on the server's directory page or website. If you can't find an answer, that is itself a finding. At the end, give a traffic-light verdict.

**Server:**_____ **Provider:**_____

**Tools that answer these questions:** official registry · MCP Inspector (shows the tools a server really offers) · mcp-scan (checks tool descriptions). *A registry listing is not a security certification.*

## 1. Who provides it?

☐ **First-party** (the vendor of the tool itself, e.g. the CRM maker) ☐ **Verified publisher / official registry entry**

☐ **Community / unknown author**

☐ Source code public? ☐ Actively maintained (recent updates, many users)?

**Red flag:** name imitates a known vendor but isn't published by them.

## 2. What can it do in your name?

☐ **Read-only** ☐ **Writes/changes data** ☐ **Irreversible actions** (send, delete, pay, publish)

What credentials does it need?_____ Can they be **scoped down** (read-only token, one mailbox, one project, not your admin account)?

## 3. Where does it run, and where does your data go?

☐ **Local** (runs on your machine, foreign code with your user's rights) ☐ **Remote** (provider's cloud, your data leaves your control)

If remote: is the provider one you already trust with this data? Is access properly authenticated (e.g. OAuth)? *Even honest vendors get this wrong.*

## 4. Does the combination become dangerous? (check the whole workflow, not one server)

Tick what your **workflow as a whole** touches:

☐ **Private data** (customer records, internal documents, mail)

☐ **Content from outsiders** (inbound e-mail, web pages, tickets, uploads, anything an attacker could write)

☐ **A channel to the outside** (can send, post, or publish)

**All three ticked = the "Lethal Trifecta".** A hidden instruction in outside content can make the agent leak private data out. **Rule of Two:** allow at most two of the three; the third only behind a human approval step.

## 5. Could you live with the worst case?

If this server were malicious or compromised tomorrow: what is the **maximum damage** with the access you gave it?_____

☐ human approval for irreversible actions ☐ minimal credentials ☐ logs you could check afterwards

## Verdict

☐ **Use it** ☐ **Use it, with these mitigations:** _____ ☐ **Not in this context**

**Reasoning (one line):**_____

## Dos and don'ts

| Do | Don't |
|---|---|
| Start with **ready-made servers** (official registry, first-party) | Connect servers you **haven't checked or don't trust** |
| Give **minimal credentials**: read-only, scoped, short-lived, the agent's own account | Hand the agent **your admin account** or put **secrets in prompts** |
| Keep a **human approval** on everything irreversible (send, delete, pay) | Let one agent read outside content, see private data **and** talk to the outside |
| **Curate per use case**: a few well-described tools | Dump **dozens of tools** into one session, the model picks wrong |
| Treat **server updates like software updates**: review what changed | Assume a server that behaved yesterday behaves tomorrow (the "rug pull") |
| Run local servers in a **container**, which limits files, network and system access | Let a local server run with **your full user rights** |
| **Keep logs** of tool calls: who, when, which tool | Notice after an incident that **nothing was recorded** |

**What you are looking for, by name:** over-broad permissions · exposed secrets · prompt injection · tool poisoning · rug pull · supply chain.

**Why this works: the model chooses tools by reading their descriptions. Those descriptions, and everything a tool returns, are text that steers the model. Vet whose text you let in.**